

K. HOVNANIAN HOMES · QA AUTOMATION

Test Automation Framework Overview

Contact Forms End-to-End Testing — Architecture & Flow Guide

FRAMEWORK

Playwright + TypeScript

1 · WHAT WE BUILT

We built a **production-grade end-to-end automation framework** for the K. Hovnanian website (khov.com) using Playwright and TypeScript. The framework validates every customer-facing contact form across four distinct surfaces — the Contact Us page, QMI detail pages, floorplan detail pages, and community detail pages. Tests run headed (visible browser) against dev, UAT, stage, or production environments with a single command, and results are published live to a ReportPortal dashboard.

The framework is designed to be **safe on production**: forms are fully filled and validated, but submission is intentionally skipped on prod so no real leads are created. All API-level assertions run only on non-prod environments where there is no risk.

2 · TECH STACK

Playwright 1.58

TypeScript 6

Node.js

Chromium (headed)

ReportPortal

dotenv

cross-env

Component	Purpose
@playwright/test	Browser automation engine — navigate, click, fill, assert, intercept network
TypeScript	Compile-time type safety for page objects, locators, and test data
dotenv / cross-env	Per-environment config — BASE_URL, env name, RP credentials
ReportPortal agent	Live test reporting dashboard with step trees, screenshots on failure

3 · ARCHITECTURE LAYERS

The framework is structured in four clean layers. Each layer has a single responsibility.

npm script

→ Entry point. Sets `TEST_ENV`, invokes Playwright CLI, passes project/file filters.

playwright.config.ts

→ Loads `environment/{env}.env` → sets `baseUrl`. Configures reporters, projects (Chrome / smoke / regression), `slowMo`, retries.

tests/baseTest.ts

→ Extends Playwright's base `test` with an `afterEach` hook that attaches a failure screenshot. All spec files import from here.

tests/*.spec.ts

→ Test specs. Orchestrate the flow: `beforeEach` navigation → test body actions → Validator assertions.

page-objects/

→ Page Object Model classes. Each page gets one class. Locators defined as `readonly Locator` fields; actions are named methods.

page-objects/basePage.ts

→ Parent class. Shared primitives: `click()`, `type()`, `navigate()`, `waitForVisible()`, popup dismissal, retry navigation.

utils/

→ `Validator` (named assertions), `apiUtils` (network intercept), `testDataUtils` (random names/emails), `constants.json` (URLs, text).

Browser (Chromium)

→ Headed, maximised window. `slowMo` 200 ms so every action is visible during the demo.

4 · RUNNING THE DEMO — STEP-BY-STEP

The command

```
npm run smoke:dev -- tests/contactForms.spec.ts
```

What happens, step by step

- 1 `npm run smoke:dev` expands to `cross-env TEST_ENV=dev npx playwright test --project=smoke`. `cross-env` injects `TEST_ENV=dev` into the process environment before Playwright starts — this works identically on macOS, Linux, and Windows.
- 2 `playwright.config.ts` loads the environment file. It reads `TEST_ENV`, picks the file `environment/dev.env`, and calls `dotenv.config()`. That file sets `BASE_URL=https://www-dev.khov.com` (and similar vars). The config then sets `use.baseURL` to that value. If `BASE_URL` is missing the config throws immediately — fail-fast.
- 3 The `smoke` project is selected by `--project=smoke`. In the config this project has `grep: /@smoke/`, so Playwright only collects tests whose description string contains the `@smoke` tag.
- 4 `-- tests/contactForms.spec.ts` (the part after `--`) is passed through as an extra Playwright argument that scopes execution to that single file.
- 5 Playwright launches Chromium headed, maximised, with `slowMo: 200` so every interaction is visible. One worker runs tests serially (no parallel browsers) to avoid contention on the live site.
- 6 For each test: `beforeEach` instantiates the page object and navigates to the target URL. The test body calls action methods on the page object, which call `BasePage` primitives. Assertions go through `Validator` which wraps each in a named `test.step()` so reports show human-readable step names.
- 7 `afterEach` (in `baseTest.ts`) runs after every test. If the test failed it captures a screenshot via `page.screenshot()` and attaches it to the test result. ReportPortal and the HTML report both display this attachment.
- 8 Results are published to the ReportPortal dashboard (if `RP_API_KEY` is set in the env file) and saved to `playwright-report/` (HTML).

Smoke tests that execute for this command

Test ID	Suite	What it verifies
TC-01	Contact Us — Field Audit	Shopping for a new home — fields, dropdown options, validation, submit
TC-01	Contact Us — Local Information	All K. Hovnanian regions appear once (no duplicates) in state dropdown
TC-02	Contact Us — Local Information	Send us a text message modal — fields, validation, fill/submit
TC-01	QMI Details	Request Information modal — fields, validation, submit
TC-01	Community Details	Request Information modal — fields, validation, submit

5 · TEST LIFECYCLE

```
// 1. Spec imports
import { test } from "../baseTest";          // always baseTest, never @playwright/test
import { expect } from "@playwright/test";    // expect imported separately
import { ContactUsPage } from "../page-objects/contactUsPage";

// 2. Describe block groups related tests
test.describe("Contact Us – Field & Dropdown Audit", () => {
  let contactUsPage: ContactUsPage;

  // 3. beforeEach: fresh navigation before every test
  test.beforeEach(async ({ page }) => {
    contactUsPage = new ContactUsPage(page);
    await contactUsPage.navigateToContactViaFooter(constants.home_page.url);
  });

  // 4. Each test: actions → Validator assertions
  test("TC-01 | Shopping for a new home @smoke", async () => {
    await contactUsPage.selectInterest("I am shopping for a new home");
    await contactUsPage.verifyFieldsForInterest("I am shopping for a new home");
    await contactUsPage.verifyRequiredFieldValidation(...);
    const resp = await contactUsPage.submitForm(...); // null on prod
    if (resp) await contactUsPage.verifySubmissionSuccess(...);
  });

  // 5. afterEach (in baseTest.ts): screenshot on failure
  // test.afterEach → page.screenshot() → testInfo.attach("failure-screenshot")
});
```

6 · PAGE OBJECT MODEL

Every page gets its own class that extends `BasePage`. Locators are declared as `readonly Locator` fields in the constructor (never hardcoded inline in tests). Methods describe user actions — `selectInterest()`, `submitForm()` — so tests read like a specification, not low-level Playwright calls.

```
// page-objects/contactUsPage.ts (simplified)
export class ContactUsPage extends BasePage {
  private readonly interestRadio: Locator;
  private readonly firstNameInput: Locator;
  private readonly submitButton: Locator;

  constructor(page: Page) {
    super(page);
    this.interestRadio = page.locator('input[name="interest"]');
    this.firstNameInput = page.getByLabel("First Name");
    this.submitButton = page.getByRole("button", { name: "Submit" });
  }

  async selectInterest(interest: ContactInterest): Promise<void> { /* ... */ }
  async verifyFieldsForInterest(interest: ContactInterest): Promise<void> { /* ... */ }
  async submitForm(interest: ContactInterest, endpoint: string): Promise<Response | null> { /* ... */ }
}
```

Shared form component

The *Request Information* modal (used on QMI, floorplan, and community pages) is extracted into its own class `RequestInformationForm`, which is composed into each page object as `this.requestInfo`. This avoids duplicating the modal verification and submission logic across multiple page objects.

7 · WHAT CONTACTFORMS.SPEC.TS COVERS

Surface	Tags	What's tested
Contact Us page (5 interest forms)	@smoke @regression	Each interest (Shopping, Mortgage, Real Estate, Subcontractor, Selling Land): fields visible, dropdown options logged, required-field error on empty submit, invalid email/phone error, fill valid data and submit.
Contact Us – Local Information	@smoke	State dropdown lists all 13 K. Hovnanian regions exactly once (no duplicates). Selecting a state reveals local contact info + "Send us a text message" CTA. Text message modal: fields, required validation, invalid validation, submit.
QMI Details page	@smoke	Request Information modal: visible, fields correct, required validation, invalid validation, fill + submit, API 200 response assertion (non-prod).
Floorplan Details page	@regression	Same as QMI — modal flow on the Clyde II floorplan at River Ranch Trails.
Community Details page	@smoke	Request Information via header CTA on River Ranch Trails community page. Same modal flow + API assertion.

8 · PRODUCTION SAFETY

No real leads are ever created on production. Every form is fully filled and all field validations are exercised on every environment — but the actual HTTP submit call is skipped on prod. The `submitForm()` / `submit()` method returns `null` on prod and the test skips the success assertion branch.

On dev and UAT, submission goes through. The framework intercepts the `/api/contact-us/` response via `waitForApi()`, asserts HTTP 200, and inspects the JSON payload to confirm the form data was correctly posted.

```
// Non-prod: submit and assert API response
const response = await qmiPage.requestInfo.submit(constants.qmi.contact_us_api);
if (response) {
  await qmiPage.requestInfo.verifyApiSubmission(response); // asserts HTTP 200 + payload
  await qmiPage.requestInfo.verifySubmissionSuccess();      // asserts success message
}
// On prod, response is null → if-block is skipped → no lead created
```

9 · REPORTING

Reporter	Output	Use case
ReportPortal	Live dashboard (browser)	Primary report. Named step tree, failure screenshots, per-launch attributes (env, OS, Node version). Enabled when <code>RP_API_KEY</code> is set.
HTML	<code>playwright-report/</code>	Local review. Full step details, traces, screenshots. Open after a run with <code>npx playwright show-report</code> .
List	Console	Real-time console output while tests run.

10 · SHARED UTILITIES

Utility	File	What it provides
Validator	<code>utils/validator.ts</code>	Static assertion helpers: <code>requireVisible</code> , <code>requireHidden</code> , <code>requireEnabled</code> , <code>requireText</code> , <code>requireUrlContains</code> , <code>requireNotEmpty</code> , <code>requireTrue</code> — each wrapped in a named <code>test.step</code> .
apiUtils	<code>utils/apiUtils.ts</code>	<code>waitForApi(page, endpoint)</code> — intercepts a network response matching the URL + HTTP 200 and returns it. Used to assert the contact-us API call on non-prod.
testDataUtils	<code>utils/testDataUtils.ts</code>	Generates randomised test data: <code>randomFirstName()</code> , <code>randomLastName()</code> , <code>randomPhone()</code> , <code>randomEmail()</code> . Keeps form submissions unique across runs.
constants.json	<code>utils/constants.json</code>	Static expected values: page URLs, headings, API paths, interest labels, state dropdown options. Single source of truth — change a URL in one place, all tests pick it up.
test_data.json	<code>utils/test_data.json</code>	Dynamic test inputs: search terms, region names, API endpoints, local-information states for the text-message modal test.